

which, in our case, is a constituency. The lists, votes and *nextPref* keep count of the first and second preference votes. The last two variables, *loser* and *listc*, are needed to process the second preference votes of the current loser. The second preference votes are added to the votes of the candidates who have not been eliminated.

```
to setup
  clear-all
  set NVoters 100
  let initListOfList []
  repeat NCandidates [
    ; add a list of N 0's as the last element of the list
    set initListOfList lput n-values NCandidates [0]
  ]
  initListOfList
end

ask patches [
  set votes n-values NCandidates [0]
  set nextPref initListOfList
  ; create a list [0 1 ... N-1]
  set listc range NCandidates
]
reset-ticks
end
```

The Main Code

In the interface, the Go button will have 'Forever' ticked.

```
to go
  vote
  tick
  if ticks = NVoters [
    compute
    stop
  ]
end
```

The vote procedure is now called. Every tick increments the counter. The loop stops when each voter has voted and the result computed.

```
to vote
  ask patches [
    ; the first NPreferred candidates get double the chance
    set candidate random (NCandidates + NPreferred) mod
    NCandidates
    ; increment the vote count of the candidate
    set votes replace-item candidate votes ( item candidate
    votes + 1 )
    ; change the colour of the patch to the current candidate
```

```
    set pcolor 20 * candidate + 5
    second_pref_vote
  ]
end

to second_pref_vote
  ; select the list for first preference candidate
  let votes2 item candidate nextPref
  ; each remaining candidate gets the same chance
  let pref2 random (NCandidates - 1)
  if pref2 >= candidate [
    set pref2 pref2 + 1
  ]
  ; increment the vote count and update the list of lists
  set votes2 replace-item pref2 votes2 ( item pref2 votes2 +
  1 )
  set nextPref replace-item candidate nextPref votes2
end
```

The 'ask patches' loops over each patch/constituency.

The methods given above just take a random vote and store the result in suitable lists.

```
to compute
  let vmin NVoters
  let vmax 0
  ask patches [
    ; index and candidate are the same for the first pref vote
    ; but as candidates are eliminated, the two will not be
    the same.

    let idx position max votes votes
    set winner item idx listc
    set vmax max list vmax max votes
    set vmin min list vmin max votes
    set pcolor 20 * winner + 5
  ]
  show (list "Min" vmin "Max" vmax)
end
```

In addition to finding the candidate with the maximum votes, the method also finds the maximum and minimum votes of the winning candidates.

Figure 2 shows the results with only the first preference votes.

Processing for the second preference

The losing candidates are eliminated, and their votes are transferred to the remaining candidates. The process is repeated until only two candidates are left.